ASSIGNMENT 3 – TASK 7

Design a Concurrent, Fault-Tolerant Processor from First Principles

Programming in Java | CO3 | 10 Marks | K5 – Evaluate

1. Problem interpretation

The examination cell must process 10,000 score files. Each expected file must be read, validated and used to calculate a result. Missing or corrupt files must be isolated so processing continues, and a final summary must report successes and failures.

2. Stage 1 – Interpret the problem (Joint)

Each file is independent: existence check, reading, parsing, validation and total calculation can run in parallel. Input files are read-only. Shared data consists of the collected result records. Failure scenarios include missing/unreadable files, malformed line count, non-numeric scores, scores outside 0–100, unexpected task errors, interruption and failure to write the final CSV.

Measurable requirements: exactly 10,000 expected names are considered; success + failure equals 10,000; one bad file does not stop other tasks; pools 1, 2, 4 and 8 are measured; each run creates a CSV audit trail with one row per expected file.

3. Stage 2 – Design (Student A)

Architecture: ResultProcessor submits ScoreTask instances to a fixed ExecutorService. ScoreTask implements Callable. Each submitted task is represented by Future. The coordinator retrieves results with Future.get(), stores them in ConcurrentLinkedQueue, sorts them and creates the summary and CSV.

```
ResultProcessor\n  -> ExecutorService (fixed pool p)\n  -> ScoreTask implements Callable<ResultRecord>\n  -> Future<ResultRecord
```

Pseudocode: create a fixed pool; submit one task for each expected filename; each task reads and validates its file and catches file/data exceptions into a failure record; retrieve every Future; shut down and await the executor; sort records; count successes and failures; write the CSV; verify success + failure = expected file count.

The concurrent queue is thread-safe and supports shared result collection without external locking. In this implementation the coordinator inserts results after Future.get(); the collection also supports safe publication by worker tasks if the design is extended.

4. Stage 3 – Implement and measure (Student B)

The implementation uses a deterministic generator to create 10,000 expected score filenames, including missing files, non-numeric scores and out-of-range values. The processor is executed with four fixed pool sizes. A small hand-check dataset has eight valid files, one corrupt file and one missing file.

5. Benchmark results

Pool	Program ms	Success	Failure	Records
1	337	425.82	9688	312
2	264	365.95	9688	312
4	227	338.18	9688	312
8	208	339.12	9688	312

These measurements are from the generation environment and are illustrative. Re-run the four pool sizes on the submission computer and replace them with your own timings and JDK/machine details.

6. Stage 4 – Defend the design (Joint)

A fixed pool bounds active threads and avoids creating one thread per file. Four workers are a reasonable starting point, but the final recommendation must follow local measurements. More threads may not help when storage throughput, metadata access, scheduling or contention becomes the bottleneck. Waiting for every Future and checking the final count ensures the coordinator accounts for every expected filename.

Remaining limitation: the implementation retains all Future references and result records until completion, so memory grows with file count. For much larger datasets, bounded submission, batching and streaming output would reduce memory use.

7. Alternative approaches and complexity

Parallel streams are concise and use the common ForkJoinPool by default. Per-file exception handling is still needed; exceptions escaping the stream can terminate the terminal operation, and pool control is less explicit. One thread per file creates 10,000 threads and high stack-memory and scheduling overhead. If S is the reserved stack size, thread stacks can approach O(10,000 × S). A fixed pool uses O(p × S) stack memory for p workers.

For F files and L score lines per file, parsing/validation is O(F × L), which is O(F) for three lines. Sorting is O(F log F), CSV writing is O(F), and Futures plus result records require O(F) memory. Ideal execution approaches O(F/p) batches, but actual speed depends on I/O and contention.

8. Joint conclusion

The fixed-thread-pool design uses Callable and Future to process independent files concurrently, isolates per-file failures, and produces a per-file audit record plus a complete summary. The measured pool size should be selected from actual machine results rather than assuming that more threads always improve performance.

9. Individual contribution record

Complete this table with your partner before submission. The assignment requires the primary responsibility and joint participation to be recorded; names and allocations are intentionally left blank rather than assumed.

Question	Student A – Primary	Student B – Primary	Joint
Q7	Design, architecture, Callable/Future, concurrency, collection,	implementation, injected failures, benchmarking	interpret requirements and defend design

10. Evidence checklist

Attach your own screenshots of source code and console output; include the four per-file CSV reports, raw benchmark table/graph, Java version, GitHub repository link, and a separate Task07 commit. Both students must execute and understand the task individually.